

We're starting out by printing the most famous computing phrase of all time! In the editor below, use either `printf` or `cout` to print the string "Hello, World!" to `stdout`.

Input Format

You do not need to read any input in this challenge.

Output Format

Print "Hello, World!" to `stdout`.

Sample Output

Hello, World!

```
1 #include<stdio.h>
2 int main()
3 {
4     printf("Hello, World!");
5 }
6
```

	Expected	Got	
✓	Hello, World!	Hello, World!	✓

Passed all tests! ✓

Activate Windows

To take a single character 'ch' as input, you can use `scanf("%c", &ch);` and `printf("%c", ch)` writes a character specified by the argument char to stdout.

`char ch;`

`scanf("%c", &ch);`

`printf("%c", ch);`

This piece of code prints the character 'ch'.

Task

You have to print the character, 'ch'.

Input Format

Take a character, 'ch' as input.

Output Format

Print the character, 'ch'

```
1 #include<stdio.h>
2 int main()
3 {
4     char ch;
5     scanf("%c",&ch);
6     printf("%c",ch);
7 }
```

	Input	Expected	Got	
✓	c	c	c	✓

Passed all tests! ✓

The `printf()` function prints the given statement to the console. The syntax is `printf("format string", argument_list);`. In the function, if we are using an integer (`%d`), character (`%c`), string (`%s`) or float (`%f`) argument, then in the format string we have to write `%d`, `%c`, `%s`, `%f` respectively.

The `scanf()` function reads the input data from the console. The syntax is `scanf("format string", argument_list);`. For example, the `scanf("%d", &number)` statement reads integer number from the console and stores the given value in variable `number`.

To input two integers separated by a space on a single line, the command is `scanf("%d %d", &n, &m)`, where `n` and `m` are the two integers.

Task

Your task is to take two numbers of `int` data type, two numbers of `float` data type as input and output their sum:

1. Declare 4 variables: two of type `int` and two of type `float`.
2. Read 2 lines of input from `stdin` (according to the sequence given in the 'Input Format' section below) and initialize your 4 variables.
3. Use the `+` and `-` operator to perform the following operations:
 - Print the sum and difference of two `int` variable on a new line.
 - Print the sum and difference of two `float` variable rounded to one decimal place on a new line.

Input Format

The first line contains two integers. The second line contains two floating point numbers.

Constraints

- $1 \leq \text{integer variables} \leq 10^4$
- $1 \leq \text{float variables} \leq 10^4$

Output Format

Print the sum and difference of both integers separated by a space on the first line, and the sum and 1 difference of both float (scaled to 1 decimal place) separated by a space on the second line.

```

1  #include<stdio.h>
2  int main()
3  {
4      int a,b;
5      float c,d;
6      scanf("%d%d",&a,&b);
7      scanf("%f %f",&c,&d);
8      printf("%d %d\n",a+b,a-b);
9      printf("%.1f %.1f",c+d,c-d);
10 }

```

	Input	Expected	Got	
✓	10 4 4.0 2.0	14 6 6.0 2.0	14 6 6.0 2.0	✓
✓	20 8 8.0 4.0	28 12 12.0 4.0	28 12 12.0 4.0	✓

Passed all tests! ✓

Write a program to input a name (as a single character) and marks of three tests as m1, m2, and m considering all the three marks have been given in integer format.

Now, you need to calculate the average of the given marks and print it along with the name as me format section.

All the test marks are in integers and hence calculate the average in integer as well. That is, you ne part of the average only and neglect the decimal part.

Input format :

Line 1 : Name(Single character)

Line 2 : Marks scored in the 3 tests separated by single space. Output

format :

First line of output prints the name of the student. Second line of the output prints the average mark.

Constraints

Marks for each student lie in the range 0 to 100 (both inclusive)

Sample Input 1 :

**A
3 4 6**

Sample Output

**A
4**

Sample Input 2 :

**T
7 3 8**

Sample Output 2 :

T

```

1 #include<stdio.h>
2 int main()
3 {
4     int a,b,c,d;
5     char ch;
6
7     scanf("%c",&ch);
8     scanf("%d%d%d",&a,&b,&c);
9     d=(a+b+c)/3;
10    printf("%c\n",ch);
11    printf("%d",d);
12 }

```

	Input	Expected	Got	
✓	A 3 4 6	A 4	A 4	✓
✓	T 7 3 8	T 6	T 6	✓
✓	R 0 100 99	R 66	R 66	✓

Passed all tests! ✓

Some C data types, their format specifiers, and their most common bit widths are as follows:

- *Int ("%d")*: 32 Bit integer
- *Long ("%ld")*: 64 bit integer
- *Char ("%c")*: Character type
- *Float ("%f")*: 32 bit real value
- *Double ("%lf")*: 64 bit real value

Reading

To read a data type, use the following syntax:

```
scanf("`format_specifier`, &val)
```

For example, to read a *character* followed by a *double*:

```
char ch; double d;
```

```
scanf("%c %lf", &ch, &d);
```

For the moment, we can ignore the spacing between format specifiers.

Printing

To print a data type, use the following syntax:

```
printf("`format_specifier`, val)
```

For example, to print a *character* followed by a *double*:

```
char ch = 'd'; double d = 234.432;
```

```
printf("%c %lf", ch, d);
```

Note: You can also use *cin* and *cout* instead of *scanf* and *printf*; however, if you are taking a million printing a million lines, it is faster to use *scanf* and *printf*.

Input Format

Input consists of the following space-separated values: *int*, *long*, *char*, *float*, and *double*, respective

Output Format

Print each element on a new line in the same order it was received as input. Note that the floating correct up to 3 decimal places and the double to 9 decimal places.

Sample Input

```
3 12345678912345 a 334.23 14049.30493
```

Sample Output

```
3
12345678912345
334.230
14049.304930000
```

Explanation

Print *int* 3,

followed by *long* 12345678912345, followed by *char* a, followed by *float* 334.23, followed by *double* 14049.30493.

```
1 #include<stdio.h>
2 int main()
3 {
4     int a;
5     long int b;
6     char ch;
7     float c;
8     double d;
9     scanf("%d %ld %c %f %lf",&a,&b,&ch,&c,&d);
10    printf("%d\n",a);
11    printf("%ld\n",b);
12    printf("%c\n",ch);
13    printf("%.3f\n",c);
14    printf("%.9lf\n",d);
15
16 }
```

	Input	Expected	Got	
✓	3 12345678912345 a 334.23 14049.30493	3 12345678912345 a 334.230 14049.304930000	3 12345678912345 a 334.230 14049.304930000	✓

Passed all tests! ✓

Write a program to print the **ASCII value** and the two adjacent characters of the given character.

Input

E

Output

69

D F

```
1  #include<stdio.h>
2  int main()
3  {
4      char ch;
5      scanf("%c",&ch);
6      printf("%d\n%c %c\n",ch,ch-1,ch+1);
7  }
```

	Input	Expected	Got	
✓	E	69 D F	69 D F	✓

Passed all tests! ✓